

HashMap – Complete Automation Guide (Excel → POI → Map → TestNG DataProvider)

A deeply simplified and intuitive HashMap guide exactly in the style engineers understand. Includes real automation usage, Excel mappings, DataProviders, REST-Assured, configuration, and now a FULLY elaborated internal working section exactly as described by Mohammed.

1■■ Why HashMap Is Important in Automation Frameworks

- Storing configuration like BaseURL, credentials, and timeouts:

```
Map config = new HashMap<>(); config.put("baseUrl", "https://app.example.com");  
config.put("username", "admin"); config.put("timeout", "30");  
System.out.println(config.get("baseUrl"));
```

- Handling Excel / JSON / CSV dynamic test data loading:

```
Map data = new HashMap<>(); data.put("username", "admin"); data.put("password", "pass123");  
data.put("role", "manager");
```

Reading Excel row → Map (simplified Apache POI snippet):

```
Map row = new HashMap<>(); row.put(sheet.getRow(0).getCell(0).getStringCellValue(),  
sheet.getRow(1).getCell(0).getStringCellValue());
```

- Feeding data into TestNG DataProvider:

```
@DataProvider(name = "loginData") public Object[][] dp() { return new Object[][] { { data } // data is a  
HashMap }; }
```

- REST-Assured headers, params, cookies:

```
Map headers = new HashMap<>(); headers.put("Content-Type", "application/json");  
headers.put("Authorization", "Bearer token123");  
given().headers(headers).when().get("/users").then().statusCode(200);
```

- Passing data between tests using Context Map:

```
Map context = new HashMap<>(); context.put("orderId", 98765); int id = (int) context.get("orderId");
```

2■■ Internal Working of HashMap (Mohammed's Deep Mental Model)

This is the FULL real-life flow of how HashMap works internally. Explained step-by-step using the actual sequence Java follows when performing put() and get().

■ PUT FLOW — When we call map.put(101, "Mohammed")

STEP 1 — HashMap receives the key (101)

HashMap calls key.hashCode(). For Integer 101 → hashCode = 101. But Java improves distribution via: hash = key.hashCode(); hash ^= (hash >>> 16);

STEP 2 — Compute bucket index

HashMap must decide which bucket (0–15) to store the key in. It computes: bucketIndex = hash & (capacity - 1); Example: capacity = 16 → mask = 15 (1111). If hash = 101 →

bucketIndex = 5 (example).

STEP 3 — Check bucket 5

Case A — Bucket empty → Create Node(hash, key, value, next=null). Case B — Bucket has nodes → Collision. Traverse chain: Node1 → Node2 → Node3 → ... Compare keys → update if match → else append new Node.

STEP 4 — Treeification (Java 8+)

If bucket chain length > 8 AND capacity ≥ 64 → Convert bucket to Red-Black Tree. This improves worst-case lookup from $O(n)$ → $O(\log n)$.

STEP 5 — Resize if load factor exceeded

If size > capacity × 0.75 → Double the capacity and redistribute entries across new buckets. This reduces collisions and improves performance.

■ GET FLOW — When we call map.get(101)

STEP 1 — Recompute hash

HashMap recomputes the exact same hash flow as put(): hash = key.hashCode(); hash ^= (hash >>> 16);

STEP 2 — Compute bucket index

Same formula: bucketIndex = hash & (capacity - 1); This ensures consistent bucket placement.

STEP 3 — Jump directly to bucket

HashMap **does NOT** search the whole array. It jumps straight to `table[bucketIndex]`.

STEP 4 — Traverse inside bucket

If bucket has 1 node → Compare keys → return value. If collision chain exists: Node1 → Node2 → Node3 → ... Traverse until key is found. If bucket is a tree → Do Red-Black Tree search ($O(\log n)$).

STEP 5 — If no match, return null

3■■■ HashMap Methods and Their Meanings

- **put(key, value)** – Add or update.
- **get(key)** – Retrieve value.
- **getOrDefault(key, default)** – Avoid null handling.
- **containsKey(key)** – Key existence check.
- **containsValue(value)** – Rare but checks value.
- **remove(key)** – Delete entry.
- **replace(key, value)** – Update only if exists.
- **putIfAbsent(key, value)** – Insert only if missing.
- **merge()** – Combine logic, useful for counters.
- **computeIfAbsent()** – Lazy initialization.

- **computeIfPresent()** – Modify only if key exists.
- **keySet()** – All keys.
- **values()** – All values.
- **entrySet()** – Iterate key-value pairs.
- **size()** – Count entries.
- **clear()** – Empty map.

4■■■ HashMap Best Interview Coding Problems

- Character frequency count
- First non-repeating character
- Two Sum
- Check Anagrams
- Find duplicates
- Count occurrences

Character Frequency: `Map map = new HashMap<>(); for (char c : s.toCharArray()) { map.put(c, map.getOrDefault(c, 0) + 1); }`

Two Sum: `Map map = new HashMap<>(); for (int i = 0; i < nums.length; i++) { int diff = target - nums[i]; if (map.containsKey(diff)) return new int[]{ map.get(diff), i }; map.put(nums[i], i); }`

5■■■ Final Takeaways

- HashMap is everywhere in automation.

- Bucket-based storage gives fast lookup.
- Treeification + resizing ensure performance.
- Essential for configuration, data loading, REST calls.
- Mastering HashMap = mastering Java Collections.